Experiment 10

Student Name: Ashutosh Yadav	UID: 23BCS11023	Branch: CSE	Section/Group: Krg-3-A
Semester: 6th	Date of Performance: 24/04/2026	Subject Name: System Design	Subject Code: 23CSH-314
Project Chosen: Online Coding Judge (LeetCode / HackerRank-like)			

1. Aim:

To design an Online Coding Judge platform (similar to LeetCode / HackerRank) that allows users to solve programming problems, run code on custom input, submit solutions, and receive verdicts with scalable and reliable architecture.

2. Objective:

- To design a scalable architecture for a coding judge system.
- To support code execution, submission evaluation, and verdict generation.
- To handle multiple concurrent submissions efficiently.
- To ensure reliability in submission processing and result storage.
- To reduce latency using caching and asynchronous processing.
- To understand real-world trade-offs in online judging platforms.

3. Tools Used:

- **Frontend (Next.js, TypeScript, Tailwind CSS)** Built the user interface for problem solving, contest pages, and code submission.
- **Editor (Monaco Editor)** Provided a coding editor experience similar to real coding platforms.
- **Backend APIs (Next.js API Routes / Node.js)** Handled problem fetching, code submission, verdict retrieval, and contest operations.
- **Execution Engine (Judge0 API)** Used for secure code execution in multiple languages and result generation.
- **Database (Convex / PostgreSQL or similar)** Stored users, problems, submissions, contests, and leaderboard data.
- **Redis** Cached frequently accessed data like problem details and contest leaderboard snapshots.
- **Message Queue (Kafka / RabbitMQ)** Used for asynchronous submission evaluation and worker communication.
- **Deployment (Cloud + Load Balancer)** Supported scalable deployment for APIs, workers, and background processing.

4. System Requirements:

A. Functional Requirements

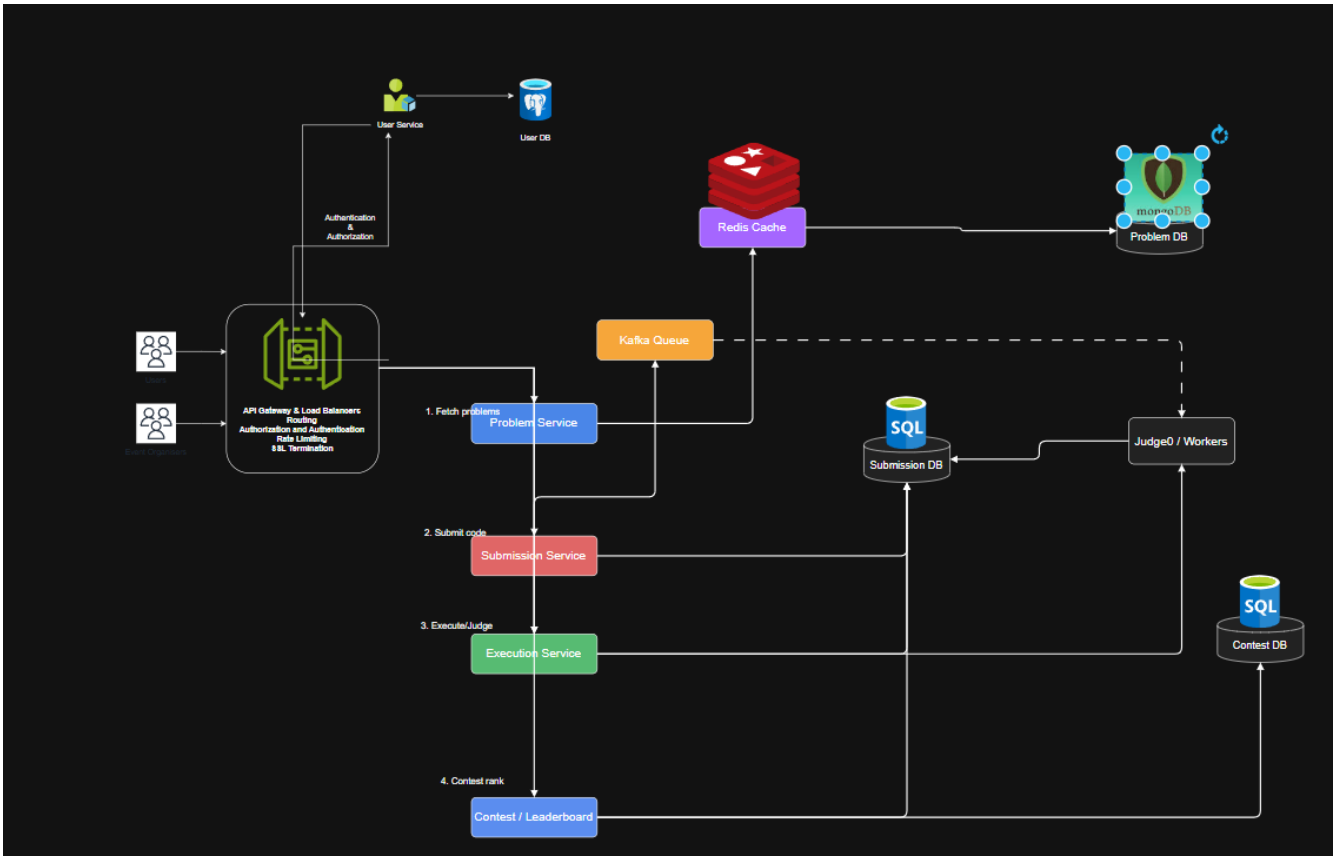
- Users should be able to register and login.
- Users should be able to browse coding problems and contests.
- Users should be able to write code and run it with custom input.
- Users should be able to submit solutions for evaluation.
- The system should generate verdicts like Accepted, Wrong Answer, TLE, and Runtime Error.

- Users should be able to view submission history and leaderboard.
- Admins should be able to add and manage problems, test cases, and contests.

B. Non-Functional Requirements

- Scalability - System should support many users and large submission spikes during contests.
- Consistency + Availability - Strong consistency is needed for submission results and leaderboard updates, while some read operations can tolerate eventual consistency.
- Latency - Problem fetch should be fast, and verdict polling should feel near real time.
- Reliability - No duplicate submission processing, and result status should remain durable even if a worker fails.

5. HLD:



6. Scalability Solution

- Use API Gateway + Load Balancer to distribute incoming traffic.
- Use Redis caching for frequently accessed problem and contest data.
- Use Kafka / Queue-based asynchronous processing for submissions.
- Use microservices / separate modules such as Problem Service, Submission Service, Execution Service, and Contest Service.
- Scale judge workers horizontally during peak submission load.
- Use database replication and indexing for read-heavy operations.
- Store code execution and result processing separately to avoid blocking user requests.

7. Learning Outcomes (What I Have Learnt)

- Understood the architecture of online coding judge platforms.
- Learned how asynchronous submission evaluation improves scalability.
- Understood the role of workers, queue, and execution engine in verdict generation.
- Learned how caching helps in problem fetch and leaderboard performance.
- Understood trade-offs between latency, consistency, and availability.
- Gained practical insight into how my Contest Compiler project can be extended into a scalable system.